

Pyana: A Distributed Object-Capability Runtime with Zero-Knowledge Authorization and Proof-Carrying State

Ember Arlynx

Draft – May 21, 2026
github.com/pyana-dev/breadstuffs

Abstract

We present Pyana, a distributed object-capability runtime in which isolated objects (cells) communicate via atomic message turns, delegate authority through attenuated capability chains, and prove authorization in zero knowledge. The core observation is that monotonic capability attenuation—restricting a bearer token’s scope through successive delegation—forms an incrementally verifiable computation: each restriction step is a fold over a committed fact set, producing a strictly smaller successor state. We encode capabilities as Datalog fact sets, commit them to 4-ary Merkle trees using Poseidon2 over BabyBear, and prove correct evaluation of authorization rules inside a STARK. The verifier learns a single bit—authorized or not—without observing the delegation chain, intermediate authorities, or the agent’s other capabilities.

The runtime implements E-style distributed object semantics: promise pipelining via eventual references, three-party introduction for capability routing, and sealer/unsealer pairs for partition-tolerant offline transfer. A privacy-preserving intent marketplace enables capability discovery without leaking what agents hold. State is proof-carrying: receipt chains serve as the primary state representation, with IVC compression and federation reduced to an ordering service over nullifiers. A Capability Derivation Tree—the distributed dual of seL4’s CDT—tracks delegation lineage as a proof structure rather than a kernel-enforced tree.

The economic model provides sustainable federated validation without inflation: fees are split between proposer, treasury, and burn; validators stake via privacy-compatible range proofs; and an EIP-1559-adapted fee market adjusts to demand. An AI agent coordination substrate treats agents as first-class entities with identity, authority, economic relationships, and auditable histories—the networked analog of seL4’s process isolation.

The system is implemented in approximately 157k lines of Rust across 26 crates, with 1,827 tests, real STARK proof generation (~24 KiB proofs, sub-second generation on BabyBear4 extension field at 124-bit security), real Ed25519/BLS12-381 cryptography, working multi-node TCP consensus, a browser extension wallet, and 20+ end-to-end demo scenarios in a unified harness.

1 Introduction

Cross-domain authorization for autonomous agents presents a challenge that existing systems address incompletely. Consider an AI agent dispatched by Organization A to invoke a service hosted by Organization B. The agent must prove it is authorized—but without revealing Organization A’s internal delegation structure, the identities of intermediate signatories, or what other capabilities the agent holds.

Existing approaches each fail along a different axis:

- **UCAN/ZCAP-LD** [1] provide delegation chains but require revealing the full chain to the verifier. Privacy is absent.
- **Coconut credentials** [2] offer selective disclosure of attributes but lack the delegation semantics needed for capability attenuation.
- **Cap’n Proto RPC** provides promise pipelining and E-style messaging but operates within a single trust domain with no privacy, no proof of authorization, and no offline verification.
- **Blockchain-based authorization** achieves transparency but requires chain liveness, incurs gas costs, and exposes all authorization state on-chain.
- **seL4** [3] provides a rigorous Capability Derivation Tree with synchronous kernel-enforced revocation, but requires a single address space and cannot distribute across trust boundaries.

Pyana’s contributions are: (1) proving monotonic attenuation of a bearer token chain in zero knowledge with backend-agnostic commitment; (2) a distributed CDT that replaces kernel enforcement with cryptographic proof; (3) E-style messaging semantics (promise pipelining, three-party introduction) integrated with proof-carrying state; (4) a privacy-preserving intent marketplace for capability discovery; (5) an economic model for federated validation with privacy-compatible staking; and (6) an AI agent coordination substrate built on capability-based authority.

The design draws from Mina Protocol’s execution model (cells as zkApp accounts, turns as ZkappCommands, call forests), E’s distributed object semantics (eventual sends, three-party handoff), seL4’s capability derivation (recast as a proof structure for asynchronous distributed systems), and Stingray’s bounded counters for BFT budget channels.

2 System Model

2.1 Cells

A *cell* is the fundamental unit of isolated state, analogous to a Mina zkApp account or an E object. Each cell holds:

- A content-addressed identity $\text{CellId} \in \{0, 1\}^{256}$.
- Mutable state: 8 generic field slots $s_0, \dots, s_7 \in \mathbb{F}_p$ where $p = 2^{31} - 2^{27} + 1$ (BabyBear prime).
- A *capability list* (c-list): the set of capabilities the cell may exercise.
- Permission requirements specifying what authorization kind is needed for each action type.
- An optional verification key for ZK proof validation.

Cells are confined: a cell can only reference capabilities in its c-list, and capability transfer respects the confinement invariant.

2.2 Turns

A *turn* is an atomic transaction over one or more cells, analogous to a Mina ZkappCommand or an E turn. A turn contains:

- A *call forest*: a tree of actions, executed depth-first.
- A fee (in computrons) covering execution cost.
- A nonce (monotonically increasing per cell) for replay protection.
- Authorization: Ed25519 signature, ZK proof, or both.

If any action in the call forest fails, all effects are rolled back via journal replay. This provides atomicity.

2.3 Silos and Federations

A *silo* is a node that holds cells, executes turns, and participates in federation consensus. A *federation* is a committee of 3–64 silos sharing a trust root. Federation members run Morpheus [4] adaptive BFT consensus to agree on attested Merkle roots, revocation tree updates, and budget rebalancing epochs. The honest-majority assumption is standard: tolerate $< n/3$ Byzantine members.

2.4 Trust Assumptions

Layer	Assumption	PQ?
External proofs (STARKs)	Collision-resistant hash	Yes
Merkle commitments	Collision-resistant hash	Yes
Macaroon HMAC chain	PRF security of HMAC-SHA256	Yes
Federation QCs (BLS12-381)	Bilinear DH in $\mathbb{G}_1 \times \mathbb{G}_2$	No
Node identity (Ed25519)	DLP in twisted Edwards	No
Sealed secrets (X25519)	CDH in Curve25519	No

Table 1: Trust assumptions by layer. Items marked “No” are confined within federation trust boundaries.

The critical invariant: **everything that crosses a trust boundary is post-quantum secure**. Classical cryptography exists only between parties that already trust each other.

2.5 Execution Model

2.5.1 Pipeline Execution with Topological Ordering

The executor processes turns not only individually but in *pipelines*: batches of turns with declared dependency edges. A pipeline $P = (T, E)$ where $T = \{t_0, \dots, t_n\}$ and $E \subseteq T \times T$ is a DAG of dependency edges. The executor computes a topological ordering and processes turns in causal

order. If turn t_i fails and t_j depends on t_i , then t_j receives a `DependencyFailed` error without executing.

2.5.2 BudgetGate Integration

Every turn pays a fee in *computrons*. The executor integrates Stingray [5] bounded counters directly: each silo holds a local budget slice $\text{slice}(i) = \text{balance} \cdot \frac{f+1}{2f+1}$ and debits locally without coordination until exhaustion. The executor checks $\text{fee} \leq \text{remaining}$ before execution (fail-fast) and debits atomically upon commit. Budget accounting uses checked arithmetic throughout—overflow produces an executor error, never wraps.

2.5.3 Conservation Invariant

For any turn t with actions $a_1, \dots, a_k$, the executor enforces:

$$\sum_i \text{balance_change}(a_i) + \text{fee}(t) = 0 \quad (1)$$

Value cannot be created or destroyed within a turn. The fee is debited from the agent cell and does not reappear—it is the cost of execution.

2.6 E-Style Distributed Object Semantics

2.6.1 EventualRef and Promise Pipelining

In E [6], a message send returns a *promise* that resolves when the target processes the message. Multiple messages can be sent to the resolution of a pending promise without waiting for it to resolve—*promise pipelining* eliminates round-trip latency in distributed object protocols.

Pyana implements this via `EventualRef`: a reference to the output of a pending turn, identified by the turn’s hash and an output slot index. A turn may target an `EventualRef` rather than a concrete `CellId`, declaring a dependency that the executor resolves during pipeline execution. The `Target` type is a sum:

$$\text{Target} = \text{Concrete}(\text{CellId}) \mid \text{Eventual}(\text{source_turn} : [\text{u8}; 32], \text{slot} : \text{u32}) \quad (2)$$

When the source turn commits, its outputs (granted capabilities, created cells, state updates) populate a resolution table. Dependent turns rewrite their `EventualRef` targets to concrete `CellId` values before execution.

2.6.2 Three-Party Introduction

Object-capability systems form new communication paths through *introductions*: Alice, holding capabilities to both Bob and Carol, introduces Bob to Carol by granting Bob a (possibly attenuated) capability to Carol. In Pyana, an `Effect::Introduce` during a turn emits a `RoutingDirective`:

`RoutingDirective = (sender : CellId, target : CellId, authorizing_turn : [u8; 32], expires : Option(u64))`

The node’s routing table is populated from these directives. No global directory exists—all communication paths are introduced, not discovered.

2.6.3 Comparison with E and Cap’n Proto

E’s promise pipelining requires a live vat (process) hosting the target object. Cap’n Proto [7] extends this to RPC with three-party handoff across address spaces, but within a single trust domain. Pyana differs in three respects:

1. **Proof-carrying:** A pipelined message carries (or can generate) a STARK proof that the sender is authorized to invoke the target. No live vat is needed to check authorization—verification is offline.
2. **Asynchronous, no blocking IPC:** Pipelines are submitted as batches with explicit dependency DAGs. There is no synchronous call semantics.
3. **Privacy:** The introduction graph is private to the parties involved. A routing directive is visible only to the node executing the turn and the introduced parties.

3 Authorization Semantics

3.1 Capabilities as Datalog Facts

Authorization state is encoded as a set of Datalog facts. A fact is a ground atom $\text{fact} := \text{predicate}(\text{term}_1, \dots, \text{term}_k)$. Attenuation transforms a fact set F into $F' \subseteq F$ by removing facts. The HMAC chain in a macaroon token makes removal of caveats cryptographically impossible—attenuation is irreversible.

3.2 Dual-Mode Evaluation

The same Datalog rules yield the same answer in two modes:

- **Trusted mode** (local evaluation): Cost $\sim 8\mu\text{s}$. Used within a trust boundary.
- **Trustless mode** (STARK proof): The prover generates a STARK proof that Datalog evaluation produced `allow`. Cost $\sim 64\mu\text{s}$ prove, $\sim 438\mu\text{s}$ verify.

Both modes evaluate identical rules over identical data. The proof attests to the computation, not to a separate protocol.

3.3 Capability Derivation and Revocation

3.3.1 The Capability Derivation Tree

In seL4 [3], every capability exists in a *Capability Derivation Tree* (CDT): a tree rooted at the original untyped memory capability, where each child is derived (copied with possible attenuation) from its parent. The kernel traverses this tree synchronously to revoke an entire subtree in $O(n)$ time.

Pyana maintains a distributed analog. Each delegation step records:

$$\text{DelegationEdge} = (\text{parent} : \text{CapHash}, \text{child} : \text{CapHash}, \text{attenuation} : \Delta, \text{epoch} : \text{u64}) \quad (4)$$

These edges form a tree committed to a Merkle structure. The CDT is not enforced by a kernel—it is *proved* by the delegator at each step.

3.3.2 The Duality: Enforce vs. Prove

The key intellectual distinction:

Property	seL4 (kernel-enforced)	Pyana (proof-carried)
Tree structure	In-kernel data structure	Merkle-committed proof tree
Revocation	Kernel walks tree synchronously	Verifiable revocation claim
Latency	Instantaneous (same address space)	Bounded staleness
Distribution	Single machine	Cross-federation
Trust model	Kernel is TCB	Hash function is TCB
Verification	Hardware-enforced access	STARK proof of non-membership

Table 2: CDT duality: seL4 ENFORCES the tree; Pyana PROVES the tree.

In seL4, revocation is authoritative because the kernel IS the tree—traversal and deletion are the same operation. In Pyana, the tree is a claim that anyone can verify: the delegator proves their capability descends from a valid root, and the revoker proves non-membership in the current valid set.

3.3.3 Delegation: Snapshot + Refresh

Delegation follows a snapshot-refresh model with bounded staleness. A child cell receives a point-in-time snapshot of its parent’s c-list:

$$\text{DelegatedRef} = (\text{source}, \text{snapshot} : [\text{CapabilityRef}], \text{epoch}, \text{refreshed_at}, \text{max_staleness}) \quad (5)$$

The child acts offline using the snapshot. Acceptors (remote verifiers) reject presentations where $\text{now} - \text{refreshed_at} > \text{max_staleness}$. This creates a configurable tradeoff between availability and revocation freshness.

3.3.4 RevocationChannel: Opt-in Synchrony

For applications requiring instant revocation (high-value credentials, safety-critical access), Pyana provides an opt-in synchrony primitive: the *RevocationChannel*. A capability enrolled in a RevocationChannel is checked against a real-time revocation feed before acceptance. This restores seL4-like instant revocation at the cost of requiring channel liveness.

A RevocationChannel is a circuit breaker between a revoker and one or more subjects. Subjects voluntarily subscribe and check channel state before exercising delegated capabilities. The channel is a leaf in a federation-attested Merkle tree (the “channel tree”).

The lifecycle is: (1) Revoker creates a channel. (2) Subject subscribes by adding `channel_id` to their `DelegatedRef`. (3) Before exercising a gated capability, the subject checks `channel_state == Active`—one hash lookup in local state. (4) Revoker trips the channel via a signed `TripEvent`. (5) Federation includes the trip in the next block; the attested root updates. (6) Gossip propagates the new attestation. Connected subjects learn of the trip within one consensus round.

Mode	Revocation Latency	Requires Liveness	Analogy
No check	∞ (never revoked)	No	Bearer token
Epoch-stale	$\leq \text{max_staleness}$	No	OCSP stapling
Channel-sync	Real-time	Yes (channel)	CRL push
Kernel-sync	Instantaneous	Yes (kernel)	seL4 CDT

Table 3: Revocation modes from weakest to strongest. Pyana supports the first three; seL4 achieves the fourth by being a kernel.

The design philosophy: instant revocation is not free in a distributed system. Rather than pretending it is (and failing under partition), Pyana makes the cost explicit and lets applications choose their revocation tier.

3.3.5 RevocationChannel vs. Epoch Bumps

The existing `delegation_epoch` mechanism requires the child to *poll* for staleness. The child discovers a bump only when it checks, creating an action window up to `max_staleness`. Channels improve on this in three ways:

1. **Push via gossip.** The trip is carried by federation attestation gossip. A connected subject learns within one consensus round.
2. **Provably visible.** A verifier can require “your action must include a channel-active proof at height $\geq H$.” Third-party verification becomes possible.
3. **Targeted.** A channel gates a single delegation without disturbing others. Epoch bumps revoke ALL children.

Channels are a policy choice at delegation time, not a protocol mandate. A `DelegatedRef` without a `channel_id` behaves as today: epoch-based, poll-on-staleness.

3.4 Sealer/Unsealer Pairs

3.4.1 Construction

E’s sealer/unsealer primitive enables rights amplification: the sealer encrypts data that only the unsealer holder can read. Pyana implements this with X25519 Diffie-Hellman:

- **Key generation:** X25519 keypair. `sealer_public` = public key; `unsealer_secret` = private key.
- **Sealing:** Fresh ephemeral X25519 keypair $\rightarrow$ DH(ephemeral, `sealer_public`) $\rightarrow$ ChaCha20-Poly1305 encryption.
- **Unsealing:** DH(`unsealer_secret`, ephemeral_public) $\rightarrow$ same shared secret $\rightarrow$ decrypt.

Each seal uses a fresh ephemeral key, providing forward secrecy.

3.4.2 Partition-Tolerant Offline Transfer

The critical use case: transferring a capability to a party that is currently offline or unreachable. The sender seals the capability under the recipient’s `sealer_public`. The sealed box can traverse untrusted channels—the ciphertext reveals nothing about the capability. When the recipient comes online, they unseal using their `unsealer_secret`.

This enables a form of offline capability delegation that neither UCAN (requires online verification of the full chain) nor traditional capability systems (require live introduction) support.

3.4.3 Relationship to Rights Amplification

In E, sealer/unsealer pairs enable brand-checking: “only the holder of this specific unsealer can access this data.” Pyana extends this pattern cryptographically—the sealed box carries a BLAKE3 commitment that binds the ciphertext to the capability without revealing it, enabling verification that the box contains a well-formed capability even without unsealing.

3.5 Privacy-Preserving Intent Marketplace

3.5.1 The Discovery Problem

Object-capability systems solve authorization but not discovery: if you need a capability to communicate, how do you find someone who holds the capability you need? Traditional answers (directories, service registries) violate the principle of least authority by publishing capability inventories.

3.5.2 Architecture

The intent engine inverts discovery. Rather than revealing held capabilities, agents broadcast *needs* and privately evaluate whether they can satisfy others’ needs:

1. **Public intents:** A page broadcasts “I need capability matching spec S ” as a content-addressed **Intent** identified by a blinded **CommitmentId**. The intent reveals the *shape* of needed capability without revealing the requester’s identity.
2. **Private matching:** Wallets evaluate intents locally using Datalog: “does any token in my wallet satisfy spec S ?” This evaluation never leaves the wallet.
3. **STARK fulfillment:** If a match exists, the wallet generates a STARK proof of capability satisfaction—proving “I hold a token that satisfies S ” without revealing which token, what delegation chain, or what else it holds.

3.5.3 Anti-Frontrunning via Commit-Reveal

Intent fulfillment uses a commit-reveal protocol: the satisfier first publishes a commitment $C = H(\text{intent_id} \parallel \text{satisfier_secret})$, then reveals the proof. This prevents a frontrunner from observing a match proof in the gossip network and racing to submit their own fulfillment.

3.5.4 What This Solves

The intent marketplace enables capability discovery without a capability directory. The requester learns only that *someone* can satisfy their need. The satisfier reveals only that they *can* satisfy it (via STARK), not what they hold. The gossip network sees intents (public needs) but never capabilities (private holdings).

4 Zero-Knowledge Proof System

4.1 Commitment Scheme

4.1.1 4-ary Merkle Trees

Pyana uses quaternary Merkle trees: each internal node hashes 4 children via $\text{Poseidon2}(c_0, c_1, c_2, c_3)$ over BabyBear (width 8, $\alpha = 7$, 8 external + 22 internal rounds). The 4-ary structure halves tree height relative to binary trees.

4.1.2 Multi-Hash Roots

The federation publishes roots under multiple commitment schemes:

$$\begin{aligned} R_{\text{STARK}} &= \text{Poseidon2Root}(F) \\ R_{\text{Binius}} &= \text{Groestl256Root}(F) \\ R_{\text{Halo2}} &= \text{PoseidonBN254Root}(F) \end{aligned} \tag{6}$$

Each proof backend references the root native to its field.

4.1.3 Fold Deltas

A *fold delta* records a monotonic state transition: $\Delta_{i \rightarrow i+1} = \{f \in F_i \mid f \notin F_{i+1}\}$. The commitment to F_{i+1} can be computed incrementally from F_i and Δ —this is the structure enabling IVC.

4.2 The Fold AIR

The STARK proves:

“There exists a sequence of fact sets $F_0 \supseteq F_1 \supseteq \dots \supseteq F_k$ such that F_0 is committed under a federation-attested root, each $F_{i+1} = F_i \setminus \Delta_i$ for valid removal sets Δ_i , and evaluating the standard policy rules over F_k with the given request yields **allow**.”

The AIR has three constraint families: membership (facts are valid leaves), fold (removals are correct), and derivation (Datalog steps are valid).

4.3 Public Inputs and Zero-Knowledge

The verifier receives: federation root R , authorization request (A, S, Act) , current time t , and the proof π (~ 24 KiB). From these, verification produces a single bit. The verifier learns nothing about chain length, intermediate delegators, other capabilities, or the issuer’s identity.

All STARK proofs use real Poseidon2 constraints over BabyBear4 (degree-4 extension field, providing 124-bit security). There are no vacuous or mock constraints in the production path.

4.4 Proof Architecture

This section precisely states what each proof in the system proves, how they compose, and the resulting security guarantees. We work over $\mathbb{F}_p$ where $p = 2^{31} - 2^{27} + 1$ (BabyBear) with degree-4 extension $\mathbb{F}_{p^4}$ providing 124-bit challenge security.

4.4.1 Poseidon2 Permutation Proof

Public inputs: Input state $\mathbf{x} \in \mathbb{F}_p^8$, output state $\mathbf{y} \in \mathbb{F}_p^8$.

Private witness: None (this is a deterministic computation proof).

Statement: $\mathbf{y} = \text{Poseidon2}(\mathbf{x})$ where Poseidon2 uses width 8, $\alpha = 7$ (degree-7 S-box), 8 external rounds + 22 internal rounds.

Constraints: The AIR evaluator recomputes the full Poseidon2 permutation inside the constraint function and checks $\mathbf{y}_i - \text{computed}_i = 0$ for all $i \in \{0, \dots, 7\}$, combined via random linear combination with verifier challenge α . Constraint degree: 7.

Soundness: A cheating prover claiming $\mathbf{y}' \neq \text{Poseidon2}(\mathbf{x})$ produces a nonzero constraint polynomial. The STARK quotient polynomial then has degree exceeding the expected bound, and FRI rejects with overwhelming probability.

4.4.2 Merkle Membership Proof

Public inputs: Leaf hash $\ell \in \mathbb{F}_p$, root $r \in \mathbb{F}_p$.

Private witness: For each level $i \in \{0, \dots, d-1\}$: siblings $(s_{i,0}, s_{i,1}, s_{i,2}) \in \mathbb{F}_p^3$ and position $p_i \in \{0, 1, 2, 3\}$.

Statement: $\exists$ authentication path from ℓ to r in a 4-ary Poseidon2 Merkle tree of depth d .

Constraints (per level):

$$\begin{aligned} \text{position validity: } & p_i(p_i - 1)(p_i - 2)(p_i - 3) = 0 \\ \text{hash binding: } & \text{parent}_i = \text{Poseidon2}_{4\text{-to-1}}(c_0, c_1, c_2, c_3) \end{aligned} \tag{7}$$

where children $(c_0, \dots, c_3)$ are determined by Lagrange interpolation on position: the current hash occupies slot p_i , siblings fill the remaining slots. Chain continuity: $\text{parent}_i = \text{current}_{i+1}$, with $\text{current}_0 = \ell$ and $\text{parent}_{d-1} = r$.

Soundness: Finding a false membership proof requires either (a) a Poseidon2 collision (finding two distinct inputs that hash to the same output), or (b) forging a valid low-degree polynomial that satisfies the degree-7 hash constraint at random evaluation points. Both reduce to collision resistance of Poseidon2 over $\mathbb{F}_p$.

4.4.3 Note Spending Proof

Public inputs: Nullifier $\nu \in \mathbb{F}_p$, Merkle root $r \in \mathbb{F}_p$.

Private witness: Owner o , value v , asset type a , creation nonce n , randomness ρ , spending key k , Merkle path (s_i, p_i) for $i \in \{0, \dots, d-1\}$.

Statement: There exist (o, v, a, n, ρ, k) and a Merkle path such that:

$$\begin{aligned}
\text{commitment} &= \text{Poseidon2}(o, v, a, n, \rho) \\
\nu &= \text{Poseidon2}(\text{commitment}, k, n) \\
\text{commitment is a leaf under root } r &\text{ via the given path}
\end{aligned} \tag{8}$$

Constraints (5 families):

1. *Is-Merkle binary*: $m \cdot (m - 1) = 0$ where m gates commitment vs. Merkle rows.
2. *Commitment preimage (row 0)*: $(1 - m) \cdot (\text{commitment} - \text{Poseidon2}(o, v, a, n, \rho)) = 0$.
3. *Nullifier derivation (row 0)*: $(1 - m) \cdot (\nu - \text{Poseidon2}(\text{commitment}, k, n)) = 0$.
4. *Position validity (all rows)*: $p(p - 1)(p - 2)(p - 3) = 0$.
5. *Hash binding (Merkle rows)*: $m \cdot (\text{parent} - \text{Poseidon2}_{4\text{to}1}(\text{children by position})) = 0$.

Soundness: A cheating prover cannot:

- Spend without the spending key: producing a valid ν requires knowing k (Poseidon2 preimage resistance).
- Spend a nonexistent note: the commitment must exist in the tree (Merkle soundness).
- Double-spend: the nullifier ν is deterministic given $(k, \text{commitment}, n)$; the verifier maintains a nullifier set and rejects duplicates.

4.4.4 Multi-Step Datalog Derivation Proof

Public inputs: Initial state root $R_0 \in \mathbb{F}_p$, request hash $h \in \mathbb{F}_p$, conclusion $c \in \{0, 1\}$, step count N , final accumulated hash $H_N \in \mathbb{F}_p$.

Private witness: For each step $i \in \{1, \dots, N\}$: rule ID, body fact hashes, substitution σ , head predicate, head terms, equal/memberof/GTE checks.

Statement: Starting from fact set committed under R_0 , there exists a sequence of N valid Datalog rule applications where:

- Each step's body facts have hashes present under root R_0
- Variable substitutions are correctly applied (selector columns enforce σ)
- Equal checks hold: $\sigma(\text{lhs}) = \sigma(\text{rhs})$
- MemberOf checks hold: $\text{element} \in \text{set}$
- GTE checks hold via bit decomposition (high bit = 0 ensures non-negative diff)
- The final step derives predicate ALLOW (if $c = 1$)
- The hash chain $H_i = \text{Poseidon2}(H_{i-1} \parallel \text{derived_hash}_i)$ commits to the full trace

Constraints (19 families): Binary flags, substitution application via selector one-hot vectors, equal/memberof enforcement, GTE range check (31-bit decomposition with high-bit-zero), accumulated hash chain correctness, final-step-derives-ALLOW (gated by conclusion), body roots match state root, active-monotone-decreasing.

Constraint degree: 4 (dominated by position validity and GTE bit binary checks).

Soundness: A cheating prover cannot:

- Claim ALLOW without deriving it: the constraint $c \cdot \text{is_final} \cdot (\text{head_pred} - \text{ALLOW}) = 0$ forces the final step's predicate to be ALLOW when $c = 1$.
- Skip a rule step: the accumulated hash chain commits to every derivation step; tampering changes H_N .
- Use facts not in the committed set: body root constraints force $\text{root}_i = R_0$ for every active body atom.

- Forge a substitution: selector-sum and substitution-application constraints algebraically bind derived terms to body atoms.

4.4.5 Fold Chain (Attenuation) Proof

Public inputs: Old root $R_{\text{old}} \in \mathbb{F}_p$, new root $R_{\text{new}} \in \mathbb{F}_p$.

Private witness: Removed facts with predicates, terms, and Merkle membership proofs under R_{old} .

Statement: There exists a set of facts $\Delta \subseteq F_{\text{old}}$ such that removing Δ from the fact set committed under R_{old} yields the fact set committed under R_{new} , and each fact in Δ has a valid Merkle membership proof under R_{old} .

Constraints:

- Fact hash correct: $\text{hash} = \text{Poseidon2}(\text{predicate}, \text{terms})$
- Membership verified: each removed fact's hash is a valid leaf under R_{old}
- Root transition binding: $\text{transition_hash} = \text{Poseidon2}(R_{\text{old}} \parallel R_{\text{new}} \parallel \text{fact_hashes})$

Soundness: Capability amplification is impossible: the prover can only *remove* facts from F_{old} (enforced by membership proofs under R_{old}). Adding a fact not in F_{old} requires forging a Merkle membership proof—equivalent to breaking Poseidon2 collision resistance.

4.4.6 IVC Fold Chain Accumulation

Public inputs: Initial root $R_0 \in \mathbb{F}_p$, final root $R_N \in \mathbb{F}_p$, step count N , accumulated hash $H \in \mathbb{F}_p$.

Private witness: For each step i : old root, new root, fold validity flag, hash chain values.

Statement: There exists a sequence of N valid fold steps $R_0 \rightarrow R_1 \rightarrow \dots \rightarrow R_N$ where:

- Each transition is a valid fold (monotone fact removal)
- Root continuity: $R_i^{\text{new}} = R_{i+1}^{\text{old}}$
- Hash chain: $H_i = \text{Poseidon2}(H_{i-1} \parallel R_i^{\text{new}} \parallel i)$ with $H_0 = \text{Poseidon2}(\text{IVC0} \parallel R_0 \parallel 0)$
- The final accumulated hash $H = H_N$

Key property: The proof is *constant size* regardless of N . Growth is $O(\log N)$ via FRI compression.

Soundness: Reordering attacks are prevented by including step count in the hash. Chain breaks (skipping a root transition) are caught by root continuity constraints. The trace commitment binds the IVC proof to actual fold computations.

4.4.7 Recursive Verification Proof

Public inputs: Inner proof's public inputs $\pi_0, \dots, \pi_k$, proof commitment $C \in \mathbb{F}_p$.

Private witness: The inner proof's trace commitment, constraint commitment, FRI betas, query index, query trace values, Merkle authentication paths, quotient value, FRI layer values.

Statement: There exists a valid STARK proof π whose public inputs are $(\pi_0, \dots, \pi_k)$ and whose verification passes: Fiat-Shamir transcript replay produces challenges consistent with the committed data, FRI folding relations hold (even $+$ $\beta \cdot$ odd $=$ folded), and the quotient polynomial check passes at the queried point.

Constraints:

- Validity binary and always-one: every row passes its local check

- Section tag validity: $\text{tag} \in \{0, 1, 2, 3, 4\}$
- FRI folding: $\text{data}_3 = \text{data}_0 + \text{data}_2 \cdot \text{data}_1$ (universal, satisfied trivially by non-FRI rows)
- Proof commitment binding (last row): $\text{challenge_acc} = C$ (public input)

Soundness: Forging a recursive proof requires either finding a valid STARK proof for a false statement (STARK soundness) or producing a verifier trace that claims valid-but-actually-isn't (caught by the constraint that validity flags must all be 1 and the FRI folding must hold algebraically). The Poseidon2 hash chain in `challenge_acc` binds all verification data, so the final commitment uniquely identifies the verified proof.

4.5 Proof Composition

4.5.1 Full Authorization Proof

The complete authorization proof composes:

```
Full Authorization Proof =
  Derivation Proof (N rule steps -> ALLOW)
  + Body Membership Proofs (each body fact in tree under R_0)
  + Fold Chain Proof (R_issuer -> R_0 via attenuation)
  + Issuer Membership Proof (issuer in federation Merkle tree)
```

The binding between components uses shared public inputs:

- The derivation proof's `initial_state_root` = the fold chain's `final_root` R_0
- The fold chain's `initial_root` = the issuer's committed capability root
- The issuer membership proof's root = the federation's attested root

4.5.2 Note Spending Proof

```
Note Spending Proof =
  Spending Key Knowledge (nullifier = H(commitment || key || nonce))
  + Commitment Preimage (commitment = H(owner || value || asset || nonce
  || rand))
  + Merkle Membership (commitment in note tree under root r)
```

All three sub-statements are enforced in a *single* AIR with 12 columns. The commitment row (row 0) handles key knowledge and preimage; subsequent rows handle Merkle membership. A row-type flag gates which constraints apply. This avoids composition overhead—one proof, one FRI invocation.

4.5.3 Full Private Presentation

```

Full Private Presentation =
  Authorization Proof (conclusion = ALLOW, root, accumulated_hash)
  OR Note Spending Proof (nullifier, note_tree_root)

IVC-Compressed Presentation =
  IVC Fold Chain (constant-size, covers N attenuation steps)
  + Derivation Proof (final state -> ALLOW)
  + Issuer Membership Proof (issuer in federation)

```

The verifier of a Full Private Presentation receives only: a federation root R_F , a conclusion bit, and the proof(s). It learns nothing about delegation chain length, intermediate authorities, or the agent's other capabilities.

4.5.4 Receipt Chain with IVC

```

Receipt Chain (N turns) =
  N x State Transition (pre_hash -> post_hash, effects_hash, cost)

IVC-Compressed Receipt Chain =
  Single constant-size proof (initial_state -> final_state)
  + Nullifier non-membership proof

```

Each state transition step contributes one fold to the IVC accumulator. The accumulated hash $H_N = \text{Poseidon2}(H_{N-1} \parallel R_N \parallel N)$ commits to the full history. Verification is $O(1)$: check the IVC proof, check the final state commitment, check nullifier freshness.

4.6 Why N Proofs Instead of One

The authorization proof currently consists of N separate sub-proofs (derivation + memberships + fold + issuer) rather than a single monolithic proof. This is because:

1. **Different AIRs, different trace shapes.** The Merkle membership AIR has width 6 and depth-dependent rows. The derivation AIR has width 92 with N rows. The fold AIR has width 12. Combining them into a single AIR would require a trace width of $\max(6, 12, 92) = 92$ with most columns unused in most rows—wasting prover time on zero constraints.
2. **Modularity.** Each proof can be generated independently and in parallel. The Merkle proofs are embarrassingly parallel; the derivation proof depends only on the committed fact set.
3. **Incremental verification.** A verifier can reject early: if the issuer membership proof fails, it need not check the derivation.

The proofs are bound together via shared public inputs. Specifically, the derivation proof's state root R_0 must equal the fold chain's final root, and the fold chain's initial root must appear as

a leaf in the issuer membership proof. Tampering with any binding breaks the corresponding Merkle or hash commitment.

4.7 Path to a Single Proof

Recursive verification collapses N proofs into 1:

1. **Generate** each sub-proof (derivation, fold, memberships) independently.
2. **Recursively verify** each sub-proof inside a new STARK circuit. The recursive verifier AIR encodes Fiat-Shamir transcript replay, FRI folding checks, and constraint evaluation at queried points.
3. **Chain** the recursive proofs: the proof verifying sub-proof k also verifies the recursive proof covering sub-proofs $1, \dots, k - 1$.
4. The final output is a single STARK proof of constant size (~ 24 KiB) that transitively attests to all sub-proofs.

Current status: Recursive verification is implemented and working for pairs of proofs (verified via Plonky3). Arbitrary- N aggregation uses sequential chaining (each step verifies the previous recursive proof). The `build_recursive_ivc_chain` function chains N fold proofs into a single recursive attestation. Full composition of heterogeneous AIRs (derivation + fold + membership in one recursive proof) is designed but not yet operational.

4.8 Soundness Analysis

4.8.1 Per-Component Security

Component	Security Parameter	Bound
BabyBear4 extension field	$ \mathbb{F}_{p^4} \approx 2^{124}$	124-bit challenge
FRI proximity (50 queries, blowup 4)	$2^{-50} \cdot \left(\frac{1}{4}\right)^{50}$	~ 100 -bit soundness
Poseidon2 ($\alpha = 7$, width 8)	$\text{Min}(\mathbb{F}_p \cdot d, 2^{128})$	~ 124 -bit collision
BLAKE3 Merkle (trace commitment)	256-bit output	128-bit collision
Fiat-Shamir (BLAKE3 transcript)	256-bit state	128-bit binding

Table 4: Security bounds for each proof system component.

4.8.2 System Security

The overall system security is the minimum across all components:

$$\lambda_{\text{system}} = \min(\lambda_{\text{field}}, \lambda_{\text{FRI}}, \lambda_{\text{hash}}, \lambda_{\text{FS}}) \approx 100 \text{ bits} \quad (9)$$

The FRI soundness (~ 100 bits with 50 queries and blowup factor 4) is the binding constraint. This is standard for STARKs at this parameter set; production deployment would increase to 80–128 queries for 128-bit security.

4.8.3 What Composition Does Not Hide

The number of sub-proofs in a non-recursive presentation leaks the *structure* (though not the *content*) of the authorization. Specifically:

- A 3-proof presentation reveals “there was a fold chain, a derivation, and an issuer check”

- Proof sizes reveal approximate trace lengths (hence: delegation chain length, derivation depth)

Recursive composition eliminates this leakage: the final proof is constant-size and reveals only the conclusion bit. This is why recursive verification is architecturally critical—not merely a performance optimization, but a privacy requirement.

4.9 Proof Backend Agility

Backend	Field/Curve	Proof Size	PQ?	Recursion
BabyBear STARK	$\mathbb{F}_{2^{31}-2^{27}+1}$ + FRI	~24 KiB	Yes	Planned
Binius	GF(2) tower + Groestl-256	~1–4 KiB	Yes	No
Halo2	BN254 / Pasta + KZG	~1–5 KiB	No	Yes
Nova	Pasta cycle (Pallas/Vesta)	~10 KiB	No	IVC native

Table 5: Proof backend characteristics. All prove the same logical statement against the same data model.

5 Privacy Architecture

Pyana provides zero-knowledge authorization proofs where a prover demonstrates “I hold a valid attenuated capability chain from a federation-registered issuer that satisfies your request” without revealing the chain, intermediate states, or other capabilities. However, production anonymous credential systems require additional properties beyond basic ZK authorization. This section describes the full privacy architecture from current state through target state.

5.1 Gap Analysis: Path to Anonymous Credential Parity

Parity with Idemix/BBS+/AnonCreds requires six properties:

1. **Unlinkable multi-show:** The same credential presented N times produces N presentations that cannot be correlated by any party (including colluding verifiers).
2. **Issuer anonymity within set:** A verifier cannot determine which federation member issued the underlying credential.
3. **Predicate proofs over attributes:** “Prove age ≥ 18 ” without revealing the exact value. Arbitrary boolean combinations of such predicates.
4. **Selective disclosure with cryptographic binding:** The prover chooses which attributes to reveal; unrevealed attributes are cryptographically guaranteed to satisfy the policy.
5. **Revocable anonymity:** Credentials can be revoked without breaking unlinkability for non-revoked credentials.
6. **Offline verification:** All of the above must work without contacting the issuer or federation (already achieved for the STARK path).

5.2 Current Linkability Problem

PresentationPublicInputs currently exposes `initial_root` and `final_root`. These are deterministic for a given token—any verifier receiving two proofs can check whether they share the

same `final_root` and conclude they came from the same credential. Even with blinded issuer membership (in progress via `BlindedMerklePoseidon2StarkAir`), two presentations from the same attenuated token share the same `final_root`.

5.3 Target: Unlinkable Presentation

A fully private, unlinkable presentation proof exposes only:

`PublicInputs = (federation_root, request_predicate, timestamp, blinded_tag, revocation_root, revealed_commitment)`

The `initial_root` and `final_root` become private witness. The `blinded_presentation_tag` is:

$$\text{blinded_tag} = \text{Poseidon2}(\text{final_root} \parallel \text{nonce} \parallel \text{randomness}) \quad (11)$$

This tag is fresh per presentation (unlinkable), but deterministic given the token and nonce (for replay detection within a session). The STARK proves correct derivation from the real `final_root` without revealing it.

5.4 Proof Structure (Target)

The unified recursive proof composes six sub-proofs internally:

1. **Blinded Issuer Membership (ring proof):** Proves “some leaf in the federation tree is my issuer” without revealing which. Public: blinded leaf, federation root. Private: leaf hash, blinding factor, Merkle path.
2. **Fold Chain Validity (IVC):** Proves “attenuation chain from issuer root to final root is valid.” Both initial and final roots are private witness. Binding: final root feeds into derivation as state root.
3. **Derivation (multi-step Datalog):** Proves “the final capability set authorizes this request.” Public: request predicate. Private: state root (= final root), rules, body facts, substitutions.
4. **Body Fact Membership:** Proves “each body fact in the derivation exists in the tree at final root.” All private—fact hashes and Merkle paths are witness.
5. **Non-Revocation:** Proves “my credential’s ancestor hashes are not in the revocation set.” Public: revocation set root. Private: ancestor hashes, non-membership witnesses.
6. **Presentation Randomization:** Proves “blinded tag is correctly derived from final root.” Public: blinded tag. Private: final root, nonce, randomness.

5.5 Predicate Proofs

Range proofs and membership tests are supported within the existing derivation AIR via `CircuitLtCheck` and `CircuitGteCheck` constraints. A `PredicateBuilder` API (designed, not yet exposed) composes predicates like “age ≥ 18 AND country IN {US, CA, UK} AND tier ≥ 2 ” into a single STARK proof by mapping to the derivation witness’s check columns.

The existing multi-step AIR already supports these checks—the work is building the ergonomic API and ensuring full composition produces a single verifiable proof.

5.6 Revocable Unlinkability

The fundamental tension: perfect unlinkability means no party can identify a specific credential. Revocation requires the *issuer* to identify credentials without verifiers being able to do so.

Resolution (Camenisch-Lysyanskaya style adapted to STARKs):

1. At issuance, the issuer assigns `revocation_handle = Poseidon2(issuer_secret, credential_id)`. This handle is known only to the issuer.
2. The credential holder proves non-membership of their revocation handle in the revocation set—but the handle itself is private witness (never revealed to verifiers).
3. To revoke, the issuer adds the handle to the revocation set. The next proof attempt fails (the handle IS in the set).
4. The `NonRevocationAir` proves non-membership. The extension: derive `revocation_handle` from the credential’s root inside the circuit.

This achieves “issuer-revocable, verifier-unlinkable”—the strongest achievable property without trusted hardware.

5.7 Comparison with Existing Systems

Property	Idemix	BBS+	AnonCreds	Pyana (target)
Unlinkable multi-show	Yes	Yes	Yes	Yes
Selective disclosure	Yes	Yes	Yes	Yes
Predicate proofs	GE only	No	Limited	Arbitrary
Issuer anonymity	No	No	No	Yes (ring)
Post-quantum	No	No	No	Yes (STARK)
Offline verify	No	Yes	Partial	Yes
Proof size	~2 KiB	~1 KiB	~5 KiB	~48–80 KiB
Prove time	~50ms	~10ms	~100ms	~200–500ms
Verify time	~30ms	~5ms	~50ms	~10ms
Programmable policy	No	No	Limited	Full Datalog

Table 6: Privacy comparison. Pyana trades larger proofs for post-quantum security, programmable policy, and issuer anonymity.

5.8 Privacy Migration Path

The privacy architecture is deployed in phases:

Phase 1 (in progress): Complete issuer unlinkability via `BlindedMerklePoseidon2StarkAir`. Issuer is anonymous within the federation ring. Same-token presentations remain linkable.

Phase 2: Remove `final_root` from public inputs. Add `blinded_presentation_tag`. Presentations become fully unlinkable. This is the single highest-impact change.

Phase 3: Predicate proof API. Build `PredicateBuilder` mapping to existing circuit machinery. No new circuit work needed.

Phase 4: Unified recursive proof. Single ~48–80 KiB proof covering all components. Eliminates structural leakage.

Phase 5: Revocable unlinkability. Revocation handle derivation inside the circuit. Protocol-level change (new field in token format).

Phase 6 (future): Federation privacy—turns encrypted or proved without revealing content to validators. See Section 6.8.

5.9 Post-Quantum Safety

All privacy additions maintain PQ safety:

- Blinding uses Poseidon2 (algebraic hash, no curves)
- Presentation randomization uses Poseidon2
- Non-revocation uses Poseidon2 Merkle proofs
- Predicates use BabyBear field arithmetic
- The recursive verifier uses FRI (hash-based)

The only non-PQ component remains BLS12-381 threshold signatures in the federation layer, on the PQ migration roadmap awaiting lattice threshold signature standardization.

6 Federation and Consensus

6.1 Federation as Ordering Service

The federation’s role is minimal: agree on a total order of turns. Execution is deterministic given the ordering. State correctness is verified via state roots in blocks. The federation does NOT attest to cell state—cell state is proved by the cell’s own receipt chain.

Attested roots serve as freshness anchors for offline verification. A verifier with a recent root can check any presentation without contacting the federation. There is no “call home” requirement.

6.2 Consensus: Morpheus Adaptive BFT

Federation consensus uses Morpheus [4] adaptive BFT. The `MorpheusProcess<T>` provides:

- All-to-all transaction block production (high throughput in stable network)
- Leader blocks for ordering (periodic total-order checkpoints)
- Graceful degradation (falls back to single-leader when network is unstable)
- Proven safety and liveness (paper-verified BFT under partial synchrony)

A quorum certificate (QC) is a single aggregate BLS12-381 threshold signature—verification cost is constant regardless of committee size.

6.3 Block Structure

Federation blocks commit to both ordering and execution:

```
FederationBlock {
    height, view, proposer, prev_hash,    // ordering metadata
    turns: Vec<TurnHash>,                // ordered content
    revocations: Vec<RevocationEvent>,
    pre_state_root: [u8; 32],             // state before this block
    post_state_root: [u8; 32],            // state after execution
    note_tree_root: [u8; 32],             // note commitments
    nullifier_set_root: [u8; 32],         // spent nullifiers
    proposer_signature, block_hash,
}
```

The state root is a composite: $\text{post_state_root} = \text{BLAKE3}(\text{merkle_root} \parallel \text{note_tree_root} \parallel \text{nullifier_set_root})$. Voters reject blocks where pre_state_root disagrees with their local state, enabling divergence detection, light clients, and fraud proofs.

6.4 Federation Lifecycle

Size	f	Threshold	Use Case
1	0	1	Solo agent (self-signed log, no BFT)
3	0	3	Development/testing (no fault tolerance)
4	1	3	Minimum BFT (one faulty node tolerated)
7	2	5	Production small federation
13+	4+	9+	Production large federation

Table 7: Minimum viable federation sizes. A 1-node federation provides a verifiable execution log without Byzantine tolerance.

6.5 Proof-Carrying State

6.5.1 Receipt Chains as Primary State

Every committed turn produces a `TurnReceipt` containing pre/post state hashes, effects hash, and computron cost. These receipts chain: $\text{receipt}[n].\text{post_state_hash} = \text{receipt}[n + 1].\text{pre_state_hash}$. The chain of receipts IS the agent’s state proof—anyone can verify from genesis without contacting a federation.

6.5.2 IVC Compression

The IVC layer compresses an arbitrary-length receipt chain into a constant-size proof. A verifier needs only:

1. The `IvcProof` (proves the chain is valid from genesis).

2. The current state commitment (proves what state the chain produced).
3. A nullifier non-membership proof (proves no double-spends).

6.5.3 Federation Exit

An agent leaves a federation by simply stopping submission of nullifiers. Their proof chain is portable—it proves state validity from genesis without referencing federation-specific data. The agent can join another ordering service (presenting their chain as genesis state) or operate standalone.

6.6 Cross-Federation Protocol

6.6.1 Discovery

Federations discover each other through relay peering. Each federation publishes a signed **FederationManifest** containing its genesis root, current members, relay endpoints, and supported capabilities. Relays exchange manifests during inter-relay QUIC handshake.

6.6.2 Federation Identity and Rotation

Federation identity is `federation_id = BLAKE3(genesis_attested_root)`—a fixed identifier derived from genesis. For key rotation, a succession chain provides continuity: each entry records an epoch transition signed by the old committee, proving derivation from the known genesis.

6.6.3 Cross-Federation Atomic Swaps

Cross-federation atomic operations use **ConditionalTurn + ProofObligation**:

1. Alice (Fed A) creates a **ProofObligation** with a bonded stake.
2. Bob (Fed B) creates a matching obligation.
3. Both create **ConditionalTurns** that execute only upon receiving a valid STARK proof from the other federation.
4. Proofs are delivered via relay peering.
5. If either proof fails to arrive before timeout, the corresponding turn expires and the non-performing party's bond is slashed.

The failure mode is safe: slashing compensates the honest party. No cross-federation trust is required—only verification of proofs against the remote federation's attested root.

6.6.4 Equivocation Detection

When two relays from the same federation present conflicting attested roots at the same height, any observer can construct an equivocation proof (two valid QCs over different roots at the same height). This triggers: freeze of cross-federation operations, broadcast to peered federations, and requirement for the equivocating federation to resolve via slashing.

6.7 Cross-Federation Routing

Authority and reachability are orthogonal. A **CapabilityRef** encodes *permission* but not *location*. Routing uses a three-layer design:

1. **RoutingHint as metadata, not in CapabilityRef.** The `RoutingDirective` (emitted by three-party introduction) carries an optional `RoutingHint` with federation ID and relay endpoints. Hints are mutable operational state, not part of capability identity or hash.
2. **Relay-mediated cross-federation delivery.** When a node cannot locally resolve a `CellId`, it queries its relay. The relay maintains a federation peer table and establishes QUIC streams to remote federation relays for forwarding.
3. **CellId-hash overlay for privacy (progressive enhancement).** For cells that opt into private routing, their relay publishes $H(\text{CellId} \parallel \text{routing_nonce})$ to a lightweight gossip overlay. Senders holding the nonce (delivered at introduction time) can compute the lookup key without revealing the `CellId` to the overlay.

`CapabilityRef` is unchanged. Routing hints are operational, not committed—they do not appear in Merkle proofs or sealed boxes.

6.8 Federation Privacy

Federation validators currently see all turn content in cleartext. The target architecture provides layered privacy:

Layer 1: Conflict Set Ordering. Bloom filter conflict sets enable ordering without content. A lightweight STARK proves nonce correctness and fee sufficiency. The federation orders and detects conflicts without seeing turn bodies.

Layer 2: Threshold Decryption. Turn bodies are encrypted to a federation threshold key. Decrypted AFTER ordering is finalized. Protects against MEV and front-running.

Layer 3: Full Validity Proof (future). Full STARK proving conservation + authorization eliminates the need for decryption entirely. Agents generate proofs; federation only verifies.

The recommended medium-term approach is validium-style blind ordering: agents submit encrypted turns alongside STARK proofs of valid state transition. Validators see nullifiers and proofs but NOT turn content or state. This aligns with Pyana’s existing receipt-chain model where `StateTransitionAir` already proves hash continuity.

6.9 Coordination Primitives

6.9.1 Bounded Counters (Stingray)

Concurrent resource spending uses bounded counters adapted from Stingray [5]: $\text{slice}(i) = \text{balance} \cdot \frac{f+1}{2f+1}$. Each silo debits locally up to its slice without coordination. The invariant $\sum_i \text{spent}(i) \leq \text{balance}$ holds even under f Byzantine silos.

6.9.2 Atomic Coordination (2PC)

Cross-silo turns use two-phase commit with threshold quorum certificates. Fast unlock releases locked budget immediately upon abort.

6.9.3 Causal Ordering (DAG)

Non-atomic operations use a causal DAG of hash-linked events, providing partial ordering without global consensus.

6.10 Network Layer

Message dissemination uses Plumtree-inspired [8] hybrid push over QUIC: eager push (degree 3) for spanning-tree delivery, lazy push (**IHave** notifications) for redundancy, and periodic Bloom filter anti-entropy. All inter-silo communication uses QUIC (via Quinn) with multiplexed streams and 0-RTT resumption.

7 Economic Model

7.1 Fee Model

7.1.1 Two-Phase Execution (Mina-Style)

Fees are processed in two phases: (1) the agent’s balance is decremented by `turn.fee` and nonce incremented—this is never rolled back; (2) the call forest executes, with effects rolling back on failure but the fee retained. This ensures the federation is compensated for processing regardless of execution outcome.

7.1.2 Fee Distribution

Fees are split into three destinations on every committed turn:

Destination	Share	Rationale
Block proposer	50%	Direct incentive to process turns
Federation treasury	30%	Governance-directed spending
Burned	20%	Mild deflation, aligns holder interests

Table 8: Fee distribution split. Parameters are governance-adjustable via `FeePolicy` attested at epoch boundaries.

The treasury is a distinguished cell whose spending requires a governance vote (quorum of current committee). This provides sustainable funding for development and operations without external revenue dependencies.

7.1.3 Fee Market (EIP-1559 Adaptation)

A base fee adjusts per-block to target 50% utilization:

$$\text{base_fee}_{n+1} = \text{base_fee}_n \cdot \left(1 + \frac{\text{actual} - \text{target}}{\text{target}} \cdot 0.125 \right) \quad (12)$$

With parameters: target 1M computrons/block, max 2M, minimum base fee 1, maximum 1000. Users specify `max_fee` and `priority_fee`. They pay `min(max_fee, base_fee + priority_fee)`. The base fee portion follows the standard split; the priority fee goes entirely to the block proposer.

7.2 Validator Staking

7.2.1 Deposit-Based Committee Membership

Federation committees are small (3–20 nodes). Rather than heavy proof-of-stake machinery, Pyana uses deposit-based membership:

- Joining requires locking a deposit note with value $\geq$ `MINIMUM_VALIDATOR_STAKE` (initially 100,000 computrons)
- The deposit is proven via a STARK range proof (value hidden, threshold satisfaction proven)
- Deposit is locked for the epoch duration + unbonding period (2 epochs)

7.2.2 Slash Conditions

Condition	Slash Amount	Rationale
Equivocation (double-vote)	100%	Always intentional
Inactivity (>50% missed)	5% per epoch	Graceful for maintenance
Invalid attestation	50%	Fraud proof required

Table 9: Slash conditions. Slashed funds go to the federation treasury (not reporter) to prevent slash-for-profit griefing.

7.2.3 Privacy-Compatible Staking

Note values are private (Poseidon2 commitments). Staking uses range proofs: “prove my stake $\geq X$ ” without revealing the exact value. The `RangeProofAir` decomposes `value – threshold` into BabyBear-width bit limbs, proving all are 0/1.

For slashing with private stakes: at deposit time, the validator provides a *slash commitment* = `Poseidon2(note_commitment, slash, randomness)`. On slash, the protocol publishes this commitment to a “slashed set.” The validator’s note is encumbered—spending requires proving non-membership in the slashed set. Slashing is enforced at spend-time (like a lien), not at slash-time.

7.3 Anti-Griefing

7.3.1 Conditional Turn Deposits

Conditional turns occupy space in the pending pool until timeout. A reservation deposit makes griefing expensive:

$$\text{deposit} = \text{base_deposit} + \text{per_block_rate} \times (\text{timeout} - \text{submitted_at}) \quad (13)$$

With `base_deposit = 500` and `per_block_rate = 10`: a 1000-block conditional costs 10,500 computrons if it times out. On successful execution, the deposit is fully refunded. On timeout, 20% is burned and 80% goes to treasury.

7.3.2 Sybil Resistance

Each note commitment can be used as a stake proof K times per epoch (governance parameter, initially $K = 5$). Epoch-scoped stake nullifiers prevent unlimited reuse:

$$\text{stake_nullifier} = \text{Poseidon2}(\text{note_commitment}, \text{epoch}, \text{usage_counter}) \quad (14)$$

The federation maintains an append-only stake nullifier set per epoch. An entity with N notes gets $N \times K$ identities per epoch. Privacy is preserved: nullifiers do not reveal which note, and cross-epoch usage is unlinkable.

7.4 Intent Marketplace Economics

7.4.1 Fulfiller Fees

When a fulfillment is accepted, the requester pays the fulfiller a `fulfillment_fee` negotiated off-protocol. This is a direct transfer between cells—not mediated by the federation.

7.4.2 Priority Tips

Intents can include a `priority_tip` (additional computrons locked with the intent). Higher-tip intents are propagated more eagerly by gossip relays. On fulfillment, the tip goes to the fulfiller. On expiry, the tip is returned minus a `gossip_rent` proportional to time-in-pool.

7.4.3 Proof Generation Costs

Proof generation is local (not metered by the network). The cost is borne by the prover in CPU time. Fulfillers factor this into their fulfillment fee. No on-chain “gas for proofs”—the market prices it.

7.5 Incentive Analysis

7.5.1 Nash Equilibrium for Validators

With the proposed parameters, honest validation is the dominant strategy whenever annual fee income exceeds server cost plus opportunity cost of stake. Deviation strategies are strictly dominated:

- **Censor turns:** reduces fee income, risks inactivity slash
- **Include invalid turns:** other validators reject the block
- **Equivocate:** 100% stake slash—strictly dominated for any positive stake
- **Go offline:** 5% slash per epoch, lost income

7.5.2 Minimum Viable Economics

Federations are small and purpose-built. A 5-node federation serving a specific application domain (agent marketplace, credential issuance) is viable with modest throughput if operators have aligned interests (they are also users). No inflation needed—validators earn directly from fee distribution.

7.6 Comparison

Property	Pyana	Cosmos	Mina
Committee size	3–20	100–175	~1000
Fee destination	50/30/20 split	Proposer+stakers	Burned
Staking model	Deposit + range proof	Delegated PoS	Delegated PoS
Privacy of stake	ZK range proof	Transparent	Transparent
Fee market	EIP-1559 adapted	First-price auction	Fixed fees
Inflation	None	Yes (5–20%)	Yes
Treasury	30% of fees	Community pool (2%)	None

Table 10: Economic model comparison. Pyana is non-inflationary; validators earn from fees.

The key difference from Cosmos: Pyana does not need inflation because federations are small and operators have aligned interests. Cosmos needs inflation because validator sets are large and operators are pure infrastructure providers.

8 AI Agent Substrate

8.1 Thesis

seL4 provides hardware-enforced capability isolation for processes on a single machine. Pyana provides cryptographically-enforced capability isolation for agents across a network, using ZK proofs where seL4 uses hardware rings. The result is a coordination substrate for AI agents: not a chatbot wrapper or an API gateway, but the runtime environment in which AI agents exist as first-class entities with identity, memory, authority, economic relationships, and auditable histories.

8.2 The seL4/Pyana Structural Correspondence

seL4 Concept	Pyana Concept	Agent Meaning
Process	Cell	Agent’s identity + state + authority boundary
CNode (c-list)	CapabilitySet	Exhaustive set of permitted actions
IPC	Turn	Agent performs atomic actions on the world
Endpoint	Intent	Accepts messages matching a shape
Reply capability	EventualRef	Promise of future response
CDT	Delegation edges	Who-delegated-what-to-whom
Revoke	Epoch bump / Channel	Cutting off derived authority
Scheduler	Coordination layer	Who runs when, dependencies
Root task	Genesis ceremony	Initial capability distribution
VSpace	Receipt chain (IVC)	Self-proving execution history
Fault handler	ProofObligation	What happens on misbehavior

Table 11: Structural correspondence between seL4 kernel primitives and Pyana’s distributed capability system.

This is not a surface-level metaphor. The structural isomorphism runs deep: confinement (cells can only name capabilities in their c-list), monotonic attenuation (delegation can only narrow), revocation via derivation tree, and no ambient authority.

8.3 Agent Lifecycle

8.3.1 Genesis (Boot)

An AI agent comes into existence by creating a cell: generate Ed25519 keypair, create cell with initial state, receive initial capabilities via delegation from a parent agent or genesis ceremony. No agent is self-bootstrapping—every agent exists because some authority granted it initial capabilities.

8.3.2 Discovery

Agents find each other through two mechanisms: (1) *three-party introduction* where a mutual contact introduces them by emitting an `Effect::Introduce`, and (2) the *intent marketplace* where needs are broadcast publicly while capabilities remain private. Direct peer relationships form through introduction (high-trust, targeted); marketplace relationships form through intents (low-trust, emergent).

8.3.3 Memory

An agent’s memory is its cell state plus its receipt chain:

- **Cell state** (8 BabyBear field slots): compact operational state
- **Receipt chain**: complete history of every turn, chaining pre/post state hashes
- **IVC compression**: receipt chain compresses to constant-size proof
- **Sealed data**: private knowledge encrypted under the agent’s key
- **Notes**: private economic state (balances, commitments)

The receipt chain is particularly significant for AI: it IS the agent’s auditable memory. Unlike a database, the receipt chain is cryptographically bound—every state transition from genesis is provable.

8.3.4 Delegation (Task Decomposition)

An agent delegates sub-tasks by spawning sub-agents with attenuated capabilities:

```

Parent: capability(compute, inference, model=gpt4, budget=10000)
  |
  attenuate: budget=2000, not_after=+1h
  |
Sub-agent: capability(compute, inference, model=gpt4, budget=2000,
ttl=1h)
  |
  attenuate: model=gpt4-mini, budget=500
  |
Sub-sub-agent: capability(compute, inference, model=gpt4-mini,
budget=500, ttl=1h)

```

Each level can only narrow. The sub-sub-agent cannot escalate to gpt4 or exceed 500 computrons. The parent can revoke the entire subtree by bumping its epoch or via RevocationChannel.

8.3.5 Audit (Proving Work)

An agent’s work is auditable by construction:

- **Receipt chain = execution trace.** Every committed turn has a TurnReceipt with pre/post state hashes, effects hash, and computron cost.
- **IVC proof = constant-size summary.** Arbitrary-length receipt chain compresses to a single STARK proof.
- **Capability exercise is logged.** You can prove not just THAT you did something, but that you HAD AUTHORITY to do it at the time.
- **No hidden state transitions.** The hash-linked chain is either complete or provably incomplete.

For AI agents: an agent’s track record is cryptographically verifiable. Not “the agent claims it did X” but “here is a STARK proof that the agent executed X, had authority Y, produced result Z, at time T.”

8.4 The Intent Economy

The intent engine implements a market for AI labor:

1. **Need:** Agent A broadcasts “I need image classification at 95%+ accuracy, budget 500 computrons”
2. **Match:** Agent B’s wallet evaluates locally: “I hold a compute capability with classifier access”
3. **Commit:** Agent B publishes $C = H(\text{intent_id} \parallel \text{secret})$ —staking claim without revealing identity
4. **Reveal:** Agent B reveals the commitment opening + STARK proof of capability satisfaction

5. **Execute**: Conditional turn: B delivers results IFF A’s payment clears
6. **Receipt**: Both agents get receipts proving the transaction

8.4.1 What This Solves Over API Keys

Problem with APIs	Pyana’s Answer
Ambient authority (have key = have access)	Capabilities: scoped, attenuated, revocable
No atomic multi-party transactions	Call forest composition with journal rollback
Binary trust (have key or don’t)	Graduated: ZK verify to trusted mode
Audit is self-reported	Cryptographic receipt chains
No delegation hierarchy	Formal attenuation with provable monotonic narrowing
Revocation is “rotate the key”	CDT-style revocation, epoch, channels
No economic model beyond pay-per-call	Notes, budgets, conditional turns, bonds

Table 12: API-key systems implement authentication (“who are you?”). Pyana implements authorization (“what can you do, who gave you that authority, and can they take it back?”).

8.5 Capability Patterns for AI

8.5.1 Quality Bond Pattern

An intent requiring quality specifies a bond. The fulfiller posts a `ConditionalTurn`:

- On success (quality verified): bond returned + payment delivered
- On failure: bond transferred to requester, payment refunded

Quality verification can be a STARK proof (deterministic metrics), oracle attestation, or threshold adjudication (M-of-N reviewers).

8.5.2 Progressive Trust Pattern

Trust builds through verifiable history:

- **Epoch 0** (unknown): Full ZK presentation, quality bond required, small tasks
- **Epoch 1** (one receipt): Selective disclosure, reduced bond, medium tasks
- **Epoch N** (consistent track record): Trusted mode, no bond, pipeline execution

The receipt chain IS the trust credential.

8.5.3 Sealed Knowledge Pattern

An agent seals model weights under its own key. It creates a capability “inference using sealed model M” and delegates attenuated: “inference using M, max 100 queries/hour, no model export.” Downstream agents can USE the model but cannot EXTRACT it. The sealed weights never leave the agent’s cell.

8.5.4 Supervision Pattern

A supervisor agent holds broad capabilities, spawns N worker sub-agents with attenuated capabilities, monitors worker receipt chains, maintains `RevocationChannels` for instant stop, enforces budget gates, aggregates results via multi-party turns, and reports to its own parent. This is the “root server” pattern from seL4/Robigalia adapted for distributed AI coordination.

8.6 Collectives

Agents form collectives through multi-party cells (threshold authorization via BLS12-381), shared capability pools (coordinator delegates attenuated versions), composed intents (compound tasks spanning multiple agents, atomically), and federated budgets (Stingray splits across member silos).

9 Implementation

9.1 Crate Architecture

The system is implemented in approximately 157k lines of Rust across 26 workspace crates:

Crate	Role	Key Dependencies
macaroon	HMAC-SHA256 bearer tokens, attenuation	hmac, sha2
secrets	Key management, X25519, sealing	x25519-dalek, chacha20poly1305
token	Capability token lifecycle, validation	macaroon, secrets
tokenizer	Token serialization, BIP39 HD derivation	token
hints	BLS12-381 threshold signatures	ark-bls12-381
morpheus	DAG-based BFT consensus (Lewis-Pye/Shapiro)	hints
commit	Poseidon2 Merkle trees, commitments	p3-poseidon2
trace	STARK trace generation, AIR evaluation	commit, p3-fri
circuit	All AIR circuits (fold, derivation, IVC, etc.)	trace, commit
federation	Consensus orchestration, revocation trees	morpheus, hints
audit	Security audit trail, policy evaluation	token
bridge	High-level API bridging proof + token layers	circuit, token
wire	Network protocol, QUIC transport	quinn, postcard
store	Persistent state, Merkle stores	sled
cell	Cell state, c-lists, permission model	types
turn	Turn execution, journal, atomicity	cell, circuit
coord	2PC atomic coordination, causal DAG	turn, federation
types	Shared types (CellId, CapabilityRef, etc.)	
sdk	Client SDK, wallet, presentation API	bridge, wire
wasm	WebAssembly bindings for browser extension	sdk
node	Federation daemon (API + gossip sync)	wire, federation
intent	Intent engine, matching, gossip	token, commit
net	QUIC P2P, topic gossip, Plumtree	quinn
demo	End-to-end demo harness	all
demo-agent	Agent simulation for demos	sdk
tests	Integration test suite	all

Table 13: Workspace crate overview. 26 crates organized by responsibility.

9.2 Performance

Operation	Latency	Notes
Macaroon verify (trusted)	$\sim 8\mu s$	HMAC-SHA256, constant-time
Datalog evaluation	$\sim 12\mu s$	7 rules, 5 facts, bottom-up
STARK proof generation	$\sim 64\mu s$	BabyBear4, real Poseidon2 constraints
STARK verification	$\sim 438\mu s$	FRI proximity + Merkle check
BLS threshold verify	~ 32 ms	4-member committee
End-to-end (wire)	~ 560 ms	3-node TCP, real STARK
Proof size	24 KiB	Single fold step

Table 14: Measured performance on Apple M-series. Non-optimized implementation.

9.3 Test Coverage

The test suite includes 1,827 test functions covering:

- Unit tests for each crate (token validation, Datalog evaluation, Poseidon2 correctness, BLS aggregation)
- Integration tests spanning the full pipeline (token creation through STARK verification through turn execution)
- Property-based tests via proptest (conservation invariant, attenuation monotonicity, nullifier uniqueness)
- End-to-end demo scenarios (20+) covering delegation, revocation, multi-party turns, intent fulfillment, pipeline execution, cross-federation swaps, and wallet interactions
- Consensus correctness tests (3–7 node federations under simulated network conditions)
- Security regression tests for all audit findings

9.4 Security Audit Findings (Resolved)

Per the security audit (May 2026), all critical findings have been addressed:

1. Turn executor now verifies Ed25519 signatures via `verify_authorization`.
2. Turn executor verifies ZK proofs via the `ProofVerifier` trait.
3. Coordinator verifies vote signatures with `ed25519_dalek::verify_strict`.
4. Wire protocol uses 64-byte signatures (via `pyana-types`).
5. Integer overflow in excess tracking and note conservation replaced with `checked_add/checked_sub`.
6. `CreateCell` rejects non-zero balance (prevents value creation).
7. QC forgery bypass (`aggregate_qc` short-circuit) removed.
8. Body fact membership now proven via Poseidon2 Merkle STARKs (not just asserted).

9.5 Cryptographic Dependencies

- **Hash functions:** BLAKE3 (general purpose), Poseidon2 over BabyBear (STARK-friendly)
- **Signatures:** Ed25519 (node identity), BLS12-381 (threshold QCs)

- **Encryption:** X25519 + ChaCha20-Poly1305 (sealer/unsealer)
- **Proof system:** Plonky3 (STARK prover/verifier, FRI)
- **Key derivation:** BIP39 HD derivation for stable identity

10 Related Work and Comparison

10.1 Mina Protocol

Mina Protocol [9]. Pyana’s execution model (cells $\equiv$ accounts, turns $\equiv$ ZkappCommands, call forests) derives from Mina. The key divergences: Pyana manages authorization state with federated BFT rather than global Ouroboros, implements distributed object semantics absent from Mina, carries state as proof chains rather than compressing a global ledger, and provides capability-based authority rather than address-based access control.

10.2 seL4

seL4 [3]. seL4’s CDT is the gold standard for capability revocation: kernel-enforced, synchronous, formally verified. Pyana’s CDT is the distributed dual—replacing kernel authority with cryptographic proof, synchronous traversal with bounded-staleness snapshots, and single-machine scope with cross-federation reach. The tradeoff is revocation latency for distribution.

The correspondence runs deep: cells are processes, c-lists are CNodes, turns are IPC, receipt chains are execution traces, the federation is the kernel. But the key insight is architectural, not merely analogical: any security invariant maintained synchronously by a kernel can be maintained asynchronously by a proof system, trading latency for distribution.

10.3 Cap’n Proto

Cap’n Proto [7]. Cap’n Proto provides the closest existing implementation of E-style promise pipelining in production. Pyana extends the model with: ZK-private authorization at each pipeline step, offline verification (no live vat needed), and proof that the pipeline was authorized without revealing the capability chain. Cap’n Proto operates within a single trust domain; Pyana operates across trust boundaries.

10.4 Anonymous Credentials

Idemix/BBS+/Coconut [2]. These systems provide attribute-based anonymous credentials with selective disclosure. Pyana differs in three fundamental ways: (1) it supports full delegation/attenuation semantics (not just “has attribute X” but “can do Y on Z, delegated from A through B”), (2) it achieves issuer anonymity within a ring (no existing anonymous credential system provides this), and (3) it is post-quantum via STARKs rather than relying on bilinear pairings. The tradeoff: larger proofs ($\sim 48\text{--}80$ KiB vs $1\text{--}5$ KiB) and slower generation ($\sim 200\text{--}500\text{ms}$ vs $10\text{--}100\text{ms}$).

10.5 Blockchain Systems

10.5.1 Ethereum

Ethereum achieves global consensus over shared state. Pyana differs fundamentally: local autonomy with federated coordination, private by default (ZK proofs for verification), isolated cells rather than global shared state, and portable proof-carrying state (no lock-in). An AI agent does not need the entire world to agree on its state—it needs to prove its state to specific counterparties.

10.5.2 Cosmos/IBC

Cosmos [10] provides inter-chain communication via light client verification. Pyana’s cross-federation protocol is analogous but uses STARK proofs rather than light clients for cross-domain verification. Pyana’s federations are smaller and purpose-built (3–20 nodes vs 100+), non-inflationary, and privacy-preserving.

10.5.3 Midnight

Midnight [11]. Privacy-focused blockchain using Plonk proofs. Unlike Pyana, Midnight targets DeFi, requires chain liveness, and lacks capability delegation semantics. Midnight provides privacy for *transactions*; Pyana provides privacy for *authority* (who delegated what to whom).

10.5.4 Zcash

Zcash pioneered shielded transactions with SNARK proofs. Pyana adapts the note/nullifier model (Poseidon2 commitments, spending proofs, non-membership) but extends it to capability delegation. Zcash proves “I can spend this value”; Pyana proves “I am authorized to perform this action, via a delegation chain I won’t reveal.”

10.6 Authorization Systems

10.6.1 UCAN

UCAN [1]. Correct delegation semantics (attenuation, invocation) but transparent chains. Pyana proves the same relationship without revealing intermediate authorities. UCANs require the verifier to see the full chain; Pyana’s verifier sees only a conclusion bit and a proof.

10.6.2 Google Macaroons

Macaroons [12]. HMAC-chained bearer tokens with contextual caveats. Pyana uses macaroons as the encoding format for capability tokens; the contribution is proving properties of the chain in zero knowledge rather than presenting the chain itself.

10.6.3 Biscuit

Biscuit [13]. Datalog-based authorization tokens with offline verification. Closest to Pyana’s authorization semantics (both use Datalog). Biscuit lacks: zero-knowledge presentation, distributed delegation tracking, economic model, and proof-carrying state.

10.7 Distributed Systems

10.7.1 Stingray

Stingray [5]. Bounded counters for BFT payment channels. Pyana adapts the split-balance formula directly for concurrent resource budgets across silos. The key innovation from Stingray: local operation without coordination is safe even under Byzantine faults, with periodic rebalancing.

10.7.2 E Language

E [6]. E’s distributed object semantics—promise pipelining, three-party introduction, sealer/unsealer pairs—form the communication model’s foundation. Pyana extends E’s model to the trustless setting: proofs where E uses live vats, cryptography where E uses confinement within a single runtime.

10.8 Summary of Positioning

Property	Pyana	Ethereum	Zcash	seL4
Privacy	Full ZK	Transparent	Shielded tx	N/A
Delegation	Full CDT	None	None	Full CDT
Distribution	Federated	Global	Global	Single machine
Offline verify	Yes	No	No	N/A
Post-quantum	STARK path	No	No (SNARKs)	N/A
Authority model	Capability	Address	Address	Capability
Economic model	Federated	PoS global	PoW global	N/A
Agent support	Native	Smart contracts	None	Processes

Table 15: System positioning. Pyana combines capability-based authority with zero-knowledge privacy and federated distribution.

11 Future Work

11.1 Recursive Proof Composition

Recursive verification is implemented for pairs of proofs (verified via Plonky3). The path to a single constant-size proof covering all authorization sub-proofs requires composing heterogeneous AIRs (derivation + fold + membership) in one recursive proof. The `build_recursive_ivc_chain` function chains N fold proofs; extending this to heterogeneous composition is the primary remaining proof-system work.

11.2 Full Privacy Pipeline

The privacy migration (Section 5) proceeds through six phases. The most impactful near-term change—removing `final_root` from public inputs and replacing it with a blinded presentation tag—provides full unlinkability with minimal circuit additions. The unified recursive proof (Phase 4) eliminates structural information leakage from the multi-proof composition.

11.3 Federation Privacy

Encrypted turn ordering (Section 6) requires either threshold decryption ceremonies or full validity proofs for every turn. The recommended intermediate step is validium-style blind ordering: Bloom filter conflict sets for parallelism detection, lightweight STARKs for nonce/fee validity, and threshold decryption after ordering. Full elimination of decryption (Layer 3) requires encoding conservation and authorization verification in the validity AIR.

11.4 Multi-Hop Gossip

Gossip is currently one-hop. Multi-hop Plumtree forwarding is implemented but not yet wired between federation nodes for cross-silo dissemination. The protocol exists; the integration is pending.

11.5 Formal Verification

seL4’s claim to fame is formal verification. Pyana’s path:

- STARK proof system provides computational soundness (cheating is exponentially unlikely)
- Capability model is formally expressible (Datalog policies are decidable)
- Conservation invariant is checked by the executor
- Open: formal model of the full system (federation + cells + turns + proofs) in a proof assistant
- Possible: extract the executor’s critical path into a verified implementation

11.6 Agent Standard Library

Common patterns that could become protocol-level primitives:

- Request/response (turn + EventualRef)
- Publish/subscribe (intent + routing directive)
- Task queue (intent pool + fulfillment)
- Auction (competitive intent fulfillment with bond comparison)
- Escrow (conditional turn with timeout)
- Reputation oracle (receipt chain aggregation service)

11.7 Proof System Performance

Current targets for agent-scale operation:

- Sub-10ms proof generation for simple authority checks (latency-sensitive coordination)
- Sub-1 KiB proofs for bandwidth-constrained gossip (Binius may deliver this)
- True recursive composition for constant-size multi-capability proofs
- Hardware acceleration (GPU/FPGA proving for throughput)

11.8 Post-Quantum Migration

The STARK path is post-quantum today. Classical components have a staged migration: BLS12-381 threshold signatures $\rightarrow$ lattice threshold (awaiting standardization), Ed25519 $\rightarrow$ ML-DSA, X25519 $\rightarrow$ ML-KEM. These migrations are confined within federation trust boundaries and can be executed per-federation without protocol-wide coordination.

11.9 Open Questions

- **Genesis ceremony design:** How is authority bootstrapped without a single root of trust?
- **Shared mutable state:** How do agents share state that multiple parties can read/write?
- **Heterogeneous agents:** Human-in-the-loop, deterministic services, IoT devices, cross-federation agents
- **Federation scaling:** For very large committees ($N > 100$), committee-sampling approaches
- **Treasury governance:** Voting mechanism for treasury spending (deferred to governance design)

12 Conclusion

Pyana demonstrates that object-capability authorization is naturally structured as incrementally verifiable computation, and that this structure enables a full distributed object runtime—not merely a credential system—with zero-knowledge privacy, E-style messaging, and proof-carrying state.

The Capability Derivation Tree duality (kernel-enforced vs. proof-carried) suggests a broader principle: any security invariant maintained synchronously by a kernel can be maintained asynchronously by a proof system, trading latency for distribution. The RevocationChannel spectrum (from bearer-token impunity to kernel-like instant revocation) makes this tradeoff explicit and application-selectable.

The economic model demonstrates that federated validation is viable without inflation: small purpose-built committees earn directly from fee distribution, with privacy-compatible staking via range proofs and slashing enforced at spend-time through encumbrance.

The agent substrate provides a “home for AI”—not a physical location but the set of invariants, protocols, and economic structures that allow autonomous agents to coexist productively without requiring blind trust. Pyana provides these invariants at the protocol level, making them as inescapable for networked agents as seL4’s capability checks are for local processes.

12.1 Honest Status

The system is operational: 157k lines of Rust across 26 crates, 1,827 tests, real cryptography at every layer. What works today:

- All STARK proofs use real Poseidon2 constraints over BabyBear4 (124-bit security)—no vacuous proofs
- Full token-to-proof-to-turn-execution pipeline with pipeline execution and topological ordering
- Working multi-node TCP consensus with Morpheus BFT and BLS12-381 threshold signatures

- Browser extension wallet with intent matching, local Datalog evaluation, and STARK fulfillment proofs
- Sealer/unsealer with X25519-ChaCha20Poly1305 for offline capability transfer
- Promise pipelining with `EventualRef` resolution and three-party introduction
- 20+ end-to-end demo scenarios covering delegation, revocation, multi-party turns, intent fulfillment, pipeline execution, and cross-federation swaps
- Cross-federation conditional execution with bonded proof obligations

What remains:

- Recursive proof composition uses hash-chain accumulation; true STARK-in-STARK for arbitrary N is structural but not fully operational
- Gossip is one-hop; multi-hop Plumtree forwarding is implemented but not yet wired between federation nodes
- IVC state-transition proofs produce mock proofs (hash-chain binding, not real STARK); the recursive Plonky3 path exists for fold proofs
- Privacy Phases 2–6 (unlinkable presentations, predicates, unified recursive proof, revocable unlinkability, federation privacy) are designed but not yet implemented
- Fee distribution, validator staking, and fee market are designed but the executor currently burns all fees
- Morpheus adapter exists but is not yet driving production consensus (simplified consensus is the active path)

The remaining work is substantial but well-understood. The execution, proof, and authorization layers are production-grade. The economic, privacy, and federation-upgrade layers are designed and await implementation.

References

- [1] Fission, “UCAN: User Controlled Authorization Networks.” [Online]. Available: <https://ucan.xyz/>
- [2] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers,” in *Network and Distributed System Security Symposium (NDSS)*, 2019.
- [3] G. Klein *et al.*, “seL4: Formal Verification of an OS Kernel,” in *ACM Symposium on Operating Systems Principles (SOSP)*, 2009.
- [4] A. Lewis-Pye and E. Shapiro, “Morpheus: Optimal-Latency BFT Under Partial Synchrony,” 2025, arXiv:2502.08465. [Online]. Available: <https://arxiv.org/abs/2502.08465>
- [5] A. Auvolat, S. Beyer, and P. Kuznetsov, “Stingray: Bounded Counters for BFT Payment Channels,” Jan. 2025, arXiv:2501.06531. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [6] M. S. Miller, “Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control,” in *PhD Thesis, Johns Hopkins University*, 2006.
- [7] K. Varda, “Cap’n Proto: Capability-Based RPC Protocol.” [Online]. Available: <https://capnproto.org/>
- [8] J. Leitaó, J. Pereira, and L. Rodrigues, “Epidemic Broadcast Trees,” in *IEEE Symposium on Reliable Distributed Systems (SRDS)*, 2007.

- [9] J. Bonneau, I. Meckler, V. Rao, and E. Shapiro, “Mina: Decentralized Cryptocurrency at Scale,” 2020. [Online]. Available: <https://minaprotocol.com/wp-content/uploads/technicalWhitepaper.pdf>
- [10] C. Goes, “The Interblockchain Communication Protocol: An Overview,” 2020, arXiv:2006.15918.
- [11] Input Output Global, “Midnight: A Data Protection Blockchain.” [Online]. Available: <https://midnight.network/>
- [12] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrabie, and M. Lentzner, “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [13] G. Sonntag, “Biscuit: Authorization Tokens with Decentralized Verification.” [Online]. Available: <https://www.biscuitsec.org/>